

Setor de Educação Profissional e Tecnológica Tecnologia em Análise e Desenvolvimento de Sistemas

DS141 - Estruturas de Dados II

02 - Análise de Algoritmos

Prof. Alex Kutzke

Maio de 2021

Roteiro

Caixeiro Viajante

Problemas Complexos

Análise de Algoritmos

Exemplo: Bubble Sort

Exemplo: Fibonacci

Conclusão

Problema do Caixeiro Viajante

Definição

Suponha que um caixeiro viajante tenha de visitar n cidades diferentes, iniciando e encerrando sua viagem na primeira cidade. Suponha, também, que não importa a ordem com que as cidades são visitadas e que de cada uma delas pode-se ir diretamente a qualquer outra. O problema do caixeiro viajante consiste em descobrir a rota que torna mínima a viagem total.

Problema do Caixeiro Viajante

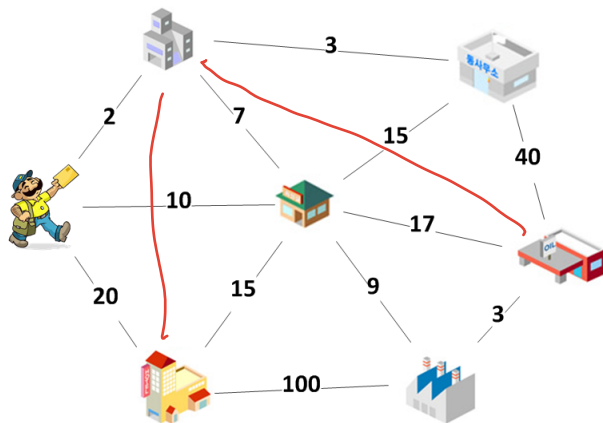


Figura: Problema do Caixeiro Viajante.

Problema do Caixeiro Viajante

Qual a solução mais simples para esse problema??

Problema do Caixeiro Viajante

Qual a solução mais simples para esse problema??
Verificar **todas as possibilidades**.

Problema do Caixeiro Viajante

Considere o problema com 4 cidades ($n = 4$): A, B, C, D .
Quais são as possibilidades?

Problema do Caixeiro Viajante

Considere o problema com 4 cidades ($n = 4$): A, B, C, D .
Quais são as possibilidades?

1. ABCDA
2. ABDCA
3. ACBDA
4. ACDBA
5. ADBCA
6. ADCBA

$$4 \quad 3! = 6$$
$$5 \quad 4! = 24$$

Problema do Caixeiro Viajante

Considere o problema com 4 cidades ($n = 4$): A, B, C, D .
Quais são as possibilidades?

1. ABCDA
2. ABDCA
3. ACBDA
4. ACDBA
5. ADBCA
6. ADCBA

E para 5 cidades, quantas rotas são possíveis? E para 6 cidades?

Problema do Caixeiro Viajante

Considere o problema com 4 cidades ($n = 4$): A, B, C, D .
Quais são as possibilidades?

1. ABCDA
2. ABDCA
3. ACBDA
4. ACDBA
5. ADBCA
6. ADCBA

E para 5 cidades, quantas rotas são possíveis? E para 6 cidades?

$$R(n) = (n - 1)!$$

Análise do Problema

- Clássico problema de **otimização combinatória**;
- Solução anterior, reduz para um **problema de enumeração** e consegue encontrar a solução ótima;

Legal! Encontramos a solução. Então por que estamos falando desse problema?

Análise do Problema

- Clássico problema de **otimização combinatória**;
- Solução anterior, reduz para um **problema de enumeração** e consegue encontrar a solução ótima;

Legal! Encontramos a solução. Então por que estamos falando desse problema?

Qual o problema dessa solução?

Análise do Problema

- Clássico problema de **otimização combinatória**;
- Solução anterior, reduz para um **problema de enumeração** e consegue encontrar a solução ótima;

Legal! Encontramos a solução. Então por que estamos falando desse problema?

Qual o problema dessa solução?

O esforço cresce em uma taxa **fatorial**!

Análise do Problema



Suponha:

- Um computador muito veloz capaz de fazer 1 bilhão de adições por segundo;
- Um problema com 20 cidades.

O computador precisa de apenas 19 adições para dizer qual o comprimento de uma rota. Portanto:

- É capaz de calcular $10^9/19 = \mathbf{53 \text{ milhões de rotas por segundo}}$;
- Ótimo, não?

Sabemos que o computador precisará calcular $19!$ rotas. Quanto é $19!$?

Análise do Problema

Suponha:

- Um computador muito veloz capaz de fazer 1 bilhão de adições por segundo;
- Um problema com 20 cidades.

O computador precisa de apenas 19 adições para dizer qual o comprimento de uma rota. Portanto:

- É capaz de calcular $10^9/19 = \mathbf{53 \text{ milhões de rotas por segundo}}$;
- Ótimo, não?

Sabemos que o computador precisará calcular $19!$ rotas. Quanto é $19!$?

121645100408832000

Análise do Problema

Suponha:

- Um computador muito veloz capaz de fazer 1 bilhão de adições por segundo;
- Um problema com 20 cidades.

O computador precisa de apenas 19 adições para dizer qual o comprimento de uma rota. Portanto:

- É capaz de calcular $10^9/19 = \mathbf{53 \text{ milhões de rotas por segundo}}$;
- Ótimo, não?

Sabemos que o computador precisará calcular $19!$ rotas. Quanto é $19!$?

121645100408832000

Assim, o computador precisará de

$$1.2 \times 10^{17} / (53 \text{ milhões}) = 2.3 \times 10^9 \text{ segundos}$$

Análise do Problema

$$2.3 \times 10^9 \text{ segundos}$$
$$= 2300000000 \text{ segundos}$$

Análise do Problema

$$\begin{aligned} &2.3 \times 10^9 \text{ segundos} \\ &= 2300000000 \text{ segundos} \\ &= 38333333 \text{ minutos} \end{aligned}$$

Análise do Problema

2.3×10^9 segundos
= 2300000000 segundos
= 38333333 minutos
= 638888 horas

Análise do Problema

2.3×10^9 segundos
= 2300000000 segundos
= 38333333 minutos
= 638888 horas
= 26620 dias

Análise do Problema

2.3×10^9 segundos
= 2300000000 segundos
= 38333333 minutos
= 638888 horas
= 26620 dias
= 73 **anos!**

Análise do Problema

$$\begin{aligned} &2.3 \times 10^9 \text{ segundos} \\ &= 2300000000 \text{ segundos} \\ &= 38333333 \text{ minutos} \\ &= 638888 \text{ horas} \\ &= 26620 \text{ dias} \\ &= 73 \text{ anos!} \end{aligned}$$

“Ah, mas tudo bem! É só alocar mais processamento. Se tivermos um computador 1000 vezes mais rápido, quanto tempo leva?”

Análise do Problema

$$\begin{aligned} &2.3 \times 10^9 \text{ segundos} \\ &= 2300000000 \text{ segundos} \\ &= 38333333 \text{ minutos} \\ &= 638888 \text{ horas} \\ &= 26620 \text{ dias} \\ &= 73 \text{ anos!} \end{aligned}$$

“Ah, mas tudo bem! É só alocar mais processamento. Se tivermos um computador 1000 vezes mais rápido, quanto tempo leva?”

$$73 \text{ anos} / 1000 = 0,073 \text{ anos} = 26 \text{ dias}$$

Análise do Problema

$$\begin{aligned} &2.3 \times 10^9 \text{ segundos} \\ &= 2300000000 \text{ segundos} \\ &= 38333333 \text{ minutos} \\ &= 638888 \text{ horas} \\ &= 26620 \text{ dias} \\ &= 73 \text{ anos!} \end{aligned}$$

“Ah, mas tudo bem! É só alocar mais processamento. Se tivermos um computador 1000 vezes mais rápido, quanto tempo leva?”

$$73 \text{ anos} / 1000 = 0,073 \text{ anos} = 26 \text{ dias}$$

Com esse novo computador e uma única cidade a mais, já levaríamos mais de um ano para calcular a melhor rota.

Análise do Problema

Tabela: Crescimento Fatorial.

n	rotas por segundo	(n - 1)!	cálculo total
5	250 milhoes	24	insignific
10	110 milhoes	362 880	0.003 seg
15	71 milhoes	87 bilhoes	20 min
20	53 milhoes	1.2×10^{17}	73 anos
25	42 milhoes	6.2×10^{23}	470 milhoes de anos

Análise do Problema

Tabela: Crescimento Polinomial

n	rotas por segundo	n^5	cálculo total
5	250 milhoes	3 125	insignific
10	110 milhoes	100 000	insignific
15	71 milhoes	759 375	0.01 seg
20	53 milhoes	3 200 000	0.06 seg
25	42 milhoes	9 765 625	0.23 seg

Problemas Complexos

E o que tudo isso quer dizer??

Problemas Complexos

E o que tudo isso quer dizer??

Que nem todos os problemas são facilmente resolvidos de maneira satisfatória por um computador.

Problemas Complexos

E o que tudo isso quer dizer??

Que nem todos os problemas são facilmente resolvidos de maneira satisfatória por um computador.

Mas não existe um algoritmo melhor?

Problemas Complexos

E o que tudo isso quer dizer??

Que nem todos os problemas são facilmente resolvidos de maneira satisfatória por um computador.

Mas não existe um algoritmo melhor?

Algoritmo ótimo? A princípio não. Se você descobrir, pode se considerar um milionário. :)

Classes de problemas

De uma maneira **muito simplificada**, podemos dividir problemas em 2 grupos:

Problemas P

Problemas resolvíveis em tempo polinomial (de acordo com o tamanho da entrada).

Problemas NP

Problemas **não** resolvíveis em tempo polinomial (de acordo com o tamanho da entrada).

- “Non-deterministic polynomial time”;
- Mas verificáveis em tempo polinomial;
- Redução de problemas;

$P = NP ??$

Análise de Algoritmos

O que a teoria de computação tem a oferecer para que possamos entender qual o tipo de problemas que estamos tratando e se seus algoritmos são bons?

Análise de Algoritmos

Estuda a correção e o desempenho de algoritmos. Em outras palavras, a análise de algoritmos procura respostas para perguntas do seguinte tipo:

- “Este algoritmo resolve o meu problema?”;
- “Quanto tempo o algoritmo consome para processar uma ‘entrada’ de tamanho n ?”.

Análise de Algoritmos

- Comparar algoritmos:
 - Qual algoritmo é melhor para uma tarefa?
- Duas abordagens:
 - Análise empírica;
 - Análise de Algoritmos.

Análise Empírica

- Bastante simples:
 - Implementa os algoritmos e testa.
- Funciona bem em alguns casos. Mas nem sempre;
- Obstáculos para a análise empírica:
 - Implementar corretamente;
 - Determinar um conjunto de dados de entrada:
 - Real;
 - Aleatório;
 - “Perverso”;
 - Dependerá da aplicação.
 - Desempenho da máquina.

Escolha de um Algoritmo

- Qual algoritmo escolher dada uma análise empírica?
- Problemas:
 - Valorizar pouco o desempenho;
 - Valorizar demais o desempenho.
- Não é possível realizar sempre uma análise empírica:
 - Demanda tempo e esforço.
- É necessário ser capaz de ter noção do desempenho do algoritmo.

Para isso, temos o estudo da **complexidade de algoritmos.**

Exemplo

Quantas vezes o seguinte algoritmo imprime "Ola mundo"?

```
1 scanf("%d",&n);  
2  
3 for(i=0; i<n; i++){  
4     printf("Ola mundo\n");  
5 }
```

n

Exemplo

E agora?

```
1 scanf("%d",&n);  
2  
3 for(i=0; i<n; i++){  
4     for(j=0; j<n; j++){  
5         printf("Ola mundo\n");  
6     }  
7 }
```

$$n \cdot n = n^2$$

Exemplo

E esse?

```
1  scanf("%d",&n);  
2  
3  for(i=0; i<n; i++){  
4      for(j=0; j<n; j++){  
5          for(k=0; k<n; k++){  
6              printf("Ola mundo\n");  
7          }  
8      }  
9  }
```

$$n \cdot n \cdot n = n^3$$

Exemplo

$$\approx \frac{n^2}{2}$$

E agora?

```
1 scanf("%d",&n);  
2  
3 for(i=0; i<n; i++){  
4     for(j=i; j<n; j++){  
5         printf("Ola mundo\n");  
6     }  
7 }
```

n
 $n-1$
 $n-2$
 $n-3$
 $\vdots$
 1

Complexidade de Algoritmos

Dizemos que, **no pior caso**, os tempos de execução dos algoritmos anteriores crescem da seguinte maneira:

$$n^2 \approx \frac{n^2}{2}$$

1. Proporcionalmente ao tamanho de n ($O(n)$);
2. Proporcionalmente ao tamanho de n^2 ($O(n^2)$);
3. Proporcionalmente ao tamanho de n^3 ($O(n^3)$);
4. Proporcionalmente ao tamanho de n^2 ($O(n^2)$);

Com estudos assim, podemos comparar algoritmos independentemente do computador utilizado.

Crescimento de Funções

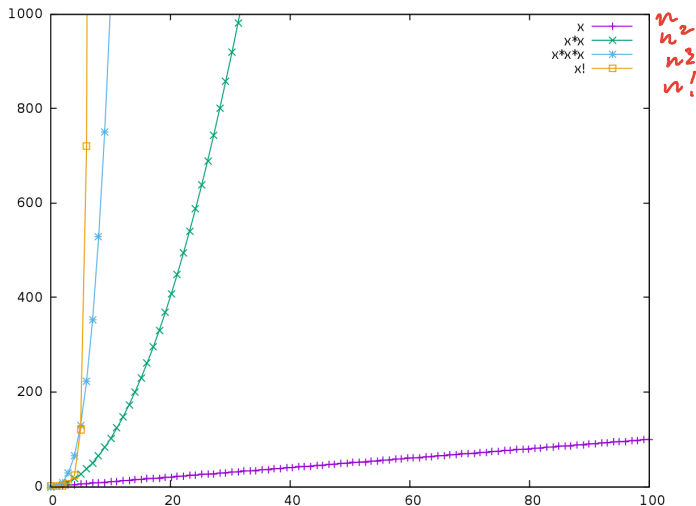


Figura: Crescimento de diferentes funções.

Exemplo: Ordenação

Problema da Ordenação

Ordenação é o ato de se colocar os elementos de uma sequência de informações, ou dados, em uma ordem predefinida.

1. Algoritmo Bubble Sort (Bolha)??

- 1.1 Fazer trocas sucessivas dois a dois até que o maior elemento esteja na última posição;
- 1.2 Repetir $n - 1$ vezes.

Bubble Sort - Análise

```
1  for(i=1; i < n; i++){
2      for(j=0; j < n - i; j++){
3          if(v[j] > v[j+1])
4              // troca v[j] com v[j+1]
5              tmp = v[j];
6              v[j] = v[j+1];
7              v[j+1] = aux;
8      }
9  }
```

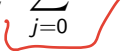
Quantas comparações entre os elementos o algoritmo Bubble Sort realiza?

Bubble Sort - Análise

```
1  for(i=1; i < n; i++){  
2      for(j=0; j < n - i; j++){  
3          if(v[j] > v[j+1]) 72  
4              // troca v[j] com v[j+1]  
5              tmp = v[j];  
6              v[j] = v[j+1];  
7              v[j+1] = aux;  
8      }  
9  }
```

Comparações: $\sum_{i=1}^{n-1} \sum_{j=0}^{n-i-1} 1$

Bubble Sort - Quantidade de Comparações

$$\sum_{i=1}^{n-1} \sum_{j=0}^{n-i-1} 1$$


Bubble Sort - Quantidade de Comparações

$$\sum_{i=1}^{n-1} \sum_{j=0}^{n-i-1} 1$$
$$= \sum_{i=1}^{n-1} (n - i)$$

Bubble Sort - Quantidade de Comparações

$$\begin{aligned} & \sum_{i=1}^{n-1} \sum_{j=0}^{n-i-1} 1 \\ &= \sum_{i=1}^{n-1} (n - i) \\ &= \underbrace{\sum_{i=1}^{n-1} n}_{n(n-1)} - \underbrace{\sum_{i=1}^{n-1} i}_{\frac{n(n-1)}{2}} \end{aligned}$$

Bubble Sort - Quantidade de Comparações

$$\begin{aligned} & \sum_{i=1}^{n-1} \sum_{j=0}^{n-i-1} 1 \\ &= \sum_{i=1}^{n-1} (n-i) \\ &= \sum_{i=1}^{n-1} n - \sum_{i=1}^{n-1} i \\ &= (n^2 - n) - \frac{(n-1)n}{2} \end{aligned}$$

Bubble Sort - Quantidade de Comparações

$$\begin{aligned}& \sum_{i=1}^{n-1} \sum_{j=0}^{n-i-1} 1 \\&= \sum_{i=1}^{n-1} (n-i) \\&= \sum_{i=1}^{n-1} n - \sum_{i=1}^{n-1} i \\&= (n^2 - n) - \frac{(n-1)n}{2} \\&= \underline{n^2 - n} - \frac{n^2 - n}{2}\end{aligned}$$

Bubble Sort - Quantidade de Comparações

$$\begin{aligned}& \sum_{i=1}^{n-1} \sum_{j=0}^{n-i-1} 1 \\&= \sum_{i=1}^{n-1} (n-i) \\&= \sum_{i=1}^{n-1} n - \sum_{i=1}^{n-1} i \\&= (n^2 - n) - \frac{(n-1)n}{2} \\&= n^2 - n - \frac{n^2 - n}{2} \\&= \frac{n^2}{2} - \frac{n}{2} \approx n^2\end{aligned}$$

Bubble Sort - Quantidade de Comparações

Por que $= \frac{n^2}{2} - \frac{n}{2} \approx n^2$?

Bubble Sort - Quantidade de Comparações

Tabela: Crescimento de diferentes funções

n	$n/2$	$n^2/2$	n^2
64	32	512	1024
128	64	2048	4096
512	256	131072	262144
1024	512	524288	1048576
2048	1024	2097152	4194304
4096	2048	8388608	16777216
8192	4096	33554432	67108864
16384	8192	134217728	268435456
32768	16384	536870912	1073741824

Bubble Sort - Quantidade de Comparações

- $\frac{n^2}{2}$ é o termo dominante quando n é grande;
- De um modo geral, podemos nos concentrar nos termos dominantes e esquecer os demais;
- Mas sempre devemos pensar quando o valor de n é grande;

”

Problema pequeno qualquer algoritmo resolve.” :)

Bubble Sort - Análise Trocas

```
1  for(i=1; i < n; i++){  
2      for(j=0; j < n - i; j++){  
3          if(v[j] > v[j+1])  
4              // troca v[j] com v[j+1]  
5              tmp = v[j];  
6              v[j] = v[j+1];  
7              v[j+1] = aux;  
8      }  
9  }
```

Handwritten notes:

- A red lightning bolt symbol is next to the `if` statement on line 3.
- A red n^2 is written next to the inner loop on line 2.
- A red bracket on the right side groups lines 4 through 7, indicating the swap operation.

Quantas trocas entre os elementos o algoritmo Bubble Sort realiza?

Bubble Sort - Análise Trocas

```
1  for(i=1; i < n; i++){
2      for(j=0; j < n - i; j++){
3          if(v[j] > v[j+1])
4              // troca v[j] com v[j+1]
5              tmp = v[j];
6              v[j] = v[j+1];
7              v[j+1] = aux;
8      }
9  }
```

1 2 3 4 5

$O(1)$

$O(n^2)$

Quantas trocas entre os elementos o algoritmo Bubble Sort realiza?

- **Melhor** caso?
- **Pior** caso?

Série de Fibonacci

0 1 1 2 3 5 8 ...

$$fib(n) = \begin{cases} 0, & \text{se } n = 0, \\ 1, & \text{se } n = 1, \\ fib(n-1) + fib(n-2), & \text{se } n \geq 2. \end{cases}$$

Algoritmo 1

```
1  long int fib(long int n)
2  {
3      if(n == 0) return(0);
4      if(n == 1) return(1);
5      return(fib(n-1) + fib(n-2));
6  }
```

Algoritmo 2

```
1  long int fib(long int n)
2  {
3      long int first = 1, second = 1, next, c, sum;
4      sum = 0;
5      for ( c = 0 ; c < n ; c++ ){
6          if ( c <= 1 )
7              next = c;
8          else{
9              next = first + second;
10             first = second;
11             second = next;
12         }
13     }
14     return(next);
15 }
```

Considerações Finais

Por que falar de tudo isso em um curso de análise e desenvolvimento?

- Sistemas resolvem problemas cada vez mais **complexos**;
 - Saber identificar se um problema tem solução computacional ou não;
 - Exemplo: Como o waze calcula o menor caminho?
 - Exemplo: BigData.
- Entender diferentes técnicas de algoritmos;
- Comparar e escolher soluções para um problema;
- Enfim, se tornar um programador melhor;

Considerações Finais

Por que falar de tudo isso em um curso de análise e desenvolvimento?

- Sistemas resolvem problemas cada vez mais **complexos**;
 - Saber identificar se um problema tem solução computacional ou não;
 - Exemplo: Como o waze calcula o menor caminho?
 - Exemplo: BigData.
- Entender diferentes técnicas de algoritmos;
- Comparar e escolher soluções para um problema;
- Enfim, se tornar um programador melhor;